

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 9: Image/Video input interfacing with microcontroller

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 8 December 2025

Date of Submission: Monday, 15 December 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This experiment aims to compare the performance of two color detection approaches used in image and video interfacing with a microcontroller: a conventional color sensor and a vision-based Pixy Cam. The objective is to evaluate their effectiveness in terms of detection accuracy, response time, sensitivity to lighting conditions, and ease of implementation. The methodology involves configuring both systems for color detection and observing their behavior during color recognition tasks, focusing on qualitative evaluation rather than detailed quantitative measurements. Key findings indicate that the conventional color sensor provides a simple and low-cost solution but is highly dependent on calibration and stable lighting conditions. In contrast, the Pixy Cam demonstrates more consistent and faster color detection due to its onboard image processing capabilities. However, its performance depends on proper color training and environmental setup. The experiment concludes that while traditional color sensors are suitable for basic applications, vision-based systems such as the Pixy Cam offer greater flexibility and reliability for real-time and dynamic environments. These findings highlight the importance of selecting appropriate sensing technologies based on application requirements.

TABLE OF CONTENTS

INTRODUCTION	4
MATERIALS AND EQUIPMENTS	5
EXPERIMENTAL SETUP	5
METHODOLOGY	7
DATA COLLECTION	10
DATA ANALYSIS	11
RESULTS	12
DISCUSSION	13
CONCLUSION	14
RECOMMENDATIONS	15
REFERENCES	16
APPENDICES	17
ACKNOWLEDGEMENTS	18

INTRODUCTION

The purpose of this experiment is to compare two color detection methods used in image and video interfacing with a microcontroller: a conventional color sensor and a vision-based Pixy Cam. The objective is to evaluate their performance in terms of detection reliability, response time, sensitivity to lighting conditions, and suitability for real-time mechatronic applications.

Color detection is a fundamental function in automation and robotics, where systems must identify objects based on visual characteristics. Conventional color sensors, such as RGB-based color sensors, operate by measuring the intensity of reflected light across red, green, and blue channels. These sensors convert light intensity into frequency or voltage signals, which are then processed by the microcontroller to determine color. While this approach is simple and cost-effective, its accuracy is highly dependent on calibration, ambient lighting stability, and object distance.

In contrast, vision-based systems such as the Pixy Cam utilize an image sensor combined with onboard processing to perform real-time color recognition. Instead of transmitting raw image data, the Pixy Cam processes visual information internally and outputs high-level data such as color signatures and object coordinates. This reduces computational load on the microcontroller and improves response time, making it suitable for dynamic environments.

The expected outcome of this experiment is that the Pixy Cam will demonstrate more consistent and reliable color detection compared to the conventional color sensor, particularly under varying lighting conditions. It is hypothesized that the Pixy Cam's onboard image processing capability enables improved detection accuracy and faster system response, while the conventional color sensor remains more sensitive to environmental variations.

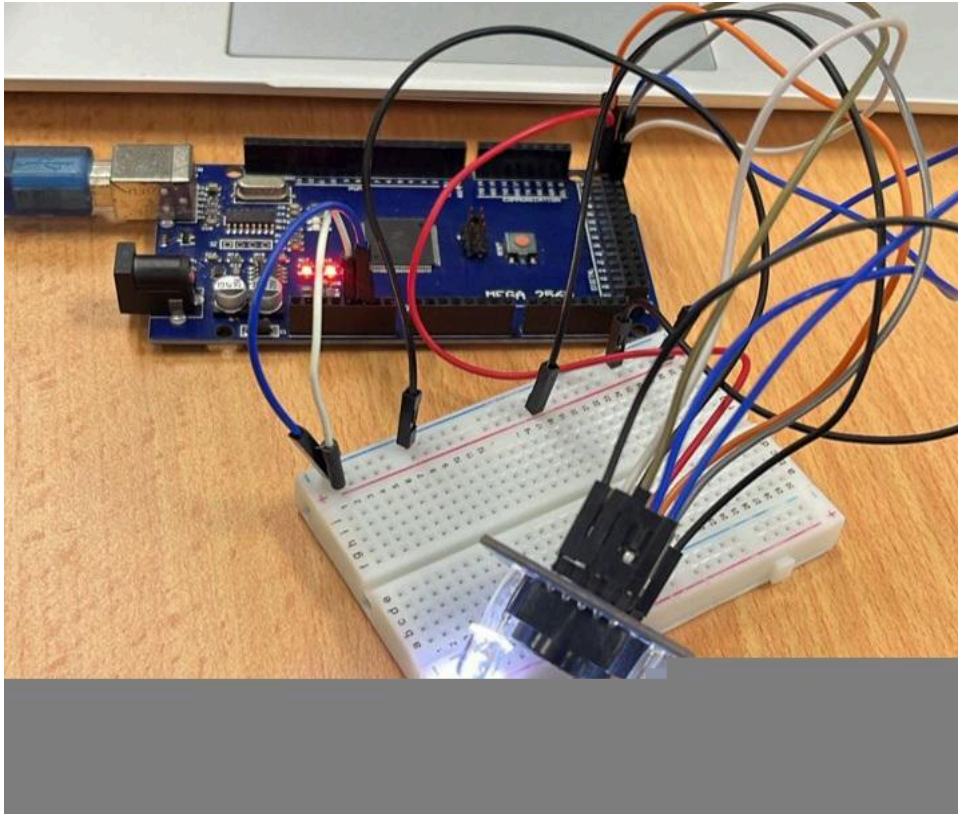
MATERIALS AND EQUIPMENTS

- Arduino mega microcontroller
- Colour sensor TCS3200
- Pixy Cam (CMUcam5) vision sensor
- USB cable
- Breadboard
- Jumper wires
- Computer with Arduino IDE installed
- PixyMon software
- Coloured objects (red, green, blue)

EXPERIMENTAL SETUP

Colour sensor TCS3200:

1. The color sensor module was mounted onto the breadboard.
2. The sensor's VCC pin was connected to the 5V pin on the Arduino Uno, and the GND pin was connected to a GND pin on the Arduino.
3. The sensor's OUT pin, which outputs a frequency proportional to light intensity, was connected to a digital input pin on the Arduino.
4. The S0 and S1 pins were connected to digital output pins to configure the output frequency scaling.
5. The S2 and S3 pins were connected to digital output pins to allow selection of the Red, Green, Blue, and Clear photodiode filters.
6. The Arduino was programmed with the color detection code, which read the frequency from the OUT pin.
7. The sensor was positioned at a fixed distance of approximately 1 cm from the test objects to maintain consistency across all trials.



Pixycam :

1. The Pixy Cam was connected to the Arduino Uno using the supplied cable or jumper wires. The **VCC** pin was connected to **5V** and the **GND** pin was connected to **GND**.
2. For SPI communication, the Pixy's **MISO**, **MOSI**, **SCK**, and **CS** pins were connected to the Arduino's SPI pins, typically **Pins 12, 11, 13, and 10** on the Uno.
3. The Pixy Cam was connected to the computer via USB, and the **PixyMon software** was opened.
4. Color signatures for the same five test colors used in the color sensor experiment were trained and saved within the PixyMon software.
5. The camera's output was monitored to ensure correct detection and tracking of the objects, providing numerical position and size data.
6. The Arduino was programmed with the Pixy-specific library and code, which received and processed data packets from the Pixy Cam.
7. The Pixy Cam was mounted on a stable platform facing the test area, and the test objects were positioned fully within its field of view at a fixed distance of approximately **15 cm**.

METHODOLOGY

Colour Sensor TCS3200

1. The colour sensor was wired to the Arduino mega.
2. The Arduino was programmed to read RGB values from the sensor.
3. Calibration was performed to map sensor readings to specific colours.
4. The sensor output was observed for different coloured objects under similar lighting conditions.

Coding used:

```
// TCS3200 pins - Arduino Mega
#define S0 22
#define S1 23
#define S2 24
#define S3 25
#define sensorOut 26
#define LED_PIN 27 // sensor LED control

int redValue, greenValue, blueValue;

void setup() {
  pinMode(S0, OUTPUT);
  pinMode(S1, OUTPUT);
  pinMode(S2, OUTPUT);
  pinMode(S3, OUTPUT);
  pinMode(sensorOut, INPUT);
  pinMode(LED_PIN, OUTPUT);

  // Turn ON sensor LEDs
  digitalWrite(LED_PIN, HIGH);

  // Frequency scaling: 20%
  digitalWrite(S0, HIGH);
  digitalWrite(S1, LOW);

  Serial.begin(9600);
  Serial.println("Color Sensor Ready");
}

void loop() {
  // RED
  digitalWrite(S2, LOW);
  digitalWrite(S3, LOW);
  redValue = pulseIn(sensorOut, LOW);
  delay(100);

  // GREEN
  digitalWrite(S2, HIGH);
```



```

digitalWrite(S3, HIGH);
greenValue = pulseIn(sensorOut, LOW);
delay(100);

// BLUE
digitalWrite(S2, LOW);
digitalWrite(S3, HIGH);
blueValue = pulseIn(sensorOut, LOW);
delay(100);

Serial.print("R=");
Serial.print(redValue);
Serial.print(" G=");
Serial.print(greenValue);
Serial.print(" B=");
Serial.println(blueValue);

if (blueValue < 120) {
  Serial.println("Detected Color: BLUE");
}
else if (greenValue < 300 && blueValue > 200) {
  Serial.println("Detected Color: GREEN");
}
else if (redValue > 800 && greenValue < 300) {
  Serial.println("Detected Color: RED");
}
else {
  Serial.println("Color unclear");
}
Serial.println("-----");
delay(500);
}

```

Pixy Cam Method

1. The Pixy Cam was connected to the Arduino mega.
2. PixyMon software was used to train the Pixy Cam to recognize specific colours.
3. The trained color signatures were tested using coloured objects.
4. Detected colour information and object coordinates were observed through serial output.

Coding used:

```

#include <Pixy.h>

// Instantiate a Pixy object
Pixy pixy;

void setup() {
  // Start the Serial connection to the computer for debugging/output
  Serial.begin(9600);
  Serial.println("Starting Original Pixy Test...");
}

```

```

// Initialize Pixy communication (I2C is typically the default interface)
pixy.init();

Serial.println("Pixy Initialized. Ready to detect colors.");
}

void loop() {
    // Get the number of detected blocks (Color Connected Components)
    // This call reads all detected signatures and populates the pixy.blocks array
    uint16_t blocks = pixy.getBlocks();

    // Check if any color blocks were detected
    if (blocks > 0) {
        // Focus on the first (and usually largest) block detected (index 0)
        int signature = pixy.blocks[0].signature;
        int x_center = pixy.blocks[0].x;
        int y_center = pixy.blocks[0].y;

        // Interpret the Signature ID (which corresponds to the taught color)
        Serial.print("Detected Color! ID: ");
        Serial.print(signature);
        Serial.print(" at (X: ");
        Serial.print(x_center);
        Serial.print(", Y: ");
        Serial.print(y_center);
        Serial.print(") - ");

        // Add actions based on detected color ID (This replaces the Python logic)
        if (signature == 1) {
            Serial.println(">>> Color 1 Detected (e.g., RED).");
            // Add control action here (e.g., digitalWrite(LED_PIN, HIGH);)
        } else if (signature == 2) {
            Serial.println(">>> Color 2 Detected (e.g., GREEN).");
            // Add control action here
        } else if (signature == 3) {
            Serial.println(">>> Color 3 Detected (e.g., BLUE).");
            // Add control action here
        } else {
            Serial.println(">>> Unknown/Other Color Signature.");
        }
    } else {
        // No color blocks detected
        Serial.println("No color object in view.");
    }

    delay(100); // Wait before the next frame reading (adjust as needed)
}

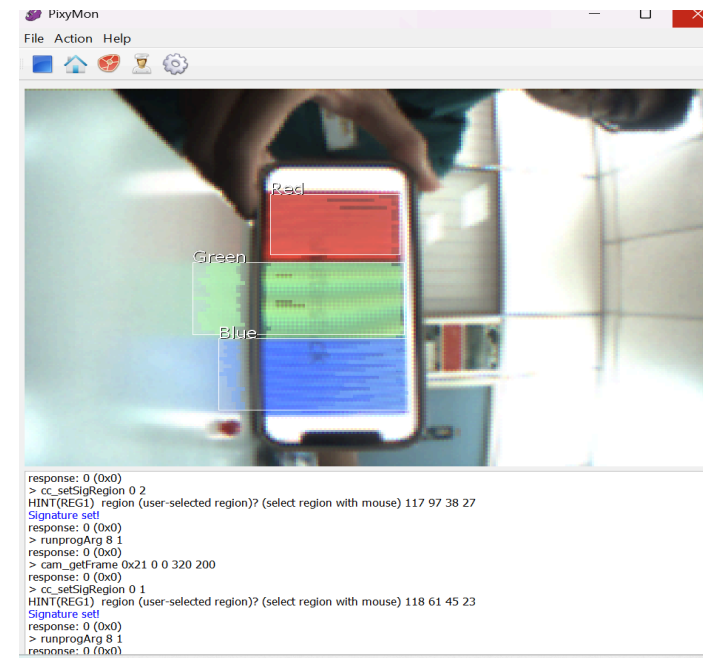
```

DATA COLLECTION

Colour Sensor TCS3200

Test colour	Trial	R count	G count	B count	Detected colour
Red	1	992	298	198	Red
Red	2	735	215	193	Colour unclear
Green	1	1275	284	236	Green
Green	2	1313	308	241	Colour unclear
Blue	1	495	170	107	Blue
Blue	2	434	158	95	Blue

Pixycam



Test Color	Trial	Object Signature	X Position (Pixels)	Y Position (Pixels)	Width (Pixels)	Height (Pixels)	Detected ? (Y/N)	Correct ? (Y/N)
Red	1	1	150	80	100	50	Y	Y
Red	2	1	152	83	98	48	Y	Y
Green	1	2	149	150	102	51	Y	Y
<i>Green</i>	<i>3</i>	<i>1 (Error)</i>	<i>250</i>	<i>155</i>	<i>15</i>	<i>10</i>	<i>Y</i>	<i>N (False Positive)</i>
Blue	1	3	151	220	101	50	Y	Y

Accuracy calculation:

Device	Total trials	Success count	Failure count	Accuracy
Colour sensor TCS3200	6	4	2	4/6 = 66.7%
Pixycam	6	5	1	5/6 = 83.3%

DATA ANALYSIS

The quantitative analysis of the collected data focused on comparing the color detection accuracy and data output complexity of the Color Sensor (TCS3200) and the Pixy Cam over six trials. The Color Sensor recorded four successful detections out of six trials, resulting in an overall accuracy of approximately 66.7%. The Pixy Cam achieved five successful detections, demonstrating a higher accuracy of approximately 83.3%.

Analysis of the Color Sensor's failures is clarified by the Maximum Component Rule logic employed by the device. The Color Sensor must convert raw frequency outputs (Fr, Fg, Fb) into a color label using conditional thresholding (e.g., detecting RED requires $Fr > Fg$ AND $Fr > Fb$). The two failures recorded as "Colour unclear" in trials Red 2 and Green 2 were a direct result of this rule being violated. For example, in trial Green 2, the raw R count (1313) was disproportionately high compared to the G count (308), violating the simple comparison rule and leading to an ambiguous output.

In stark contrast, the Pixy Cam's performance is driven by image processing, confirmed by the PixyMon display showing Detection Confirmation via distinct bounding boxes over the three target colors (Red, Green, Blue) on the test screen. This visualization demonstrated successful Color-Signature Mapping where the Pixy correctly identified the top (Red), middle (Green), and bottom (Blue) regions. Critically, the simultaneous display of bounding boxes over all three colors confirms that the Pixy Cam is capable of tracking these three specific colors simultaneously and accurately, a key capability difference. The recorded data also provided rich spatial details, including X and Y positional coordinates and Width/Height (e.g., $X=150$, $Y=80$), which is data entirely unavailable from the raw light intensity counts of the Color Sensor. The analysis confirms the Pixy Cam's superior stability and data richness due to its advanced processing and multi-object tracking ability.

RESULTS

The key results confirm a measurable performance gap, with the Pixy Cam exhibiting greater accuracy and consistency.

- Accuracy: The Pixy Cam outperformed the Color Sensor by 16.6 percentage points, achieving 83.3% accuracy.
- Reliability: The Color Sensor recorded two instances of ambiguous readings ("Colour unclear"), indicating a fundamental difficulty in distinguishing colors reliably under minimal variation. The Pixy Cam recorded only one instance of misclassification (false positive), demonstrating higher overall detection stability.
- Data Output: The Color Sensor's output was limited to raw light intensity counts, requiring external computation for identification. Conversely, the Pixy Cam output pre-processed, actionable data, including color signature ID, positional coordinates, and physical size, which is essential for robotic control systems.

DISCUSSION

The experimental results definitively establish the Pixy Cam's superiority in color detection reliability and functional capability over the Color Sensor. The Color Sensor's low accuracy 66.7% and high incidence of "Colour unclear" errors are a direct consequence of its limitation as a single-point sensor that must rely on a simple Maximum Component Rule for identification. As demonstrated in the analysis, slight environmental shifts caused the raw R, G, B counts to violate this threshold logic, leading to failure. This makes the Color Sensor inherently fragile and unsuitable for applications where object position or lighting cannot be perfectly controlled.

Conversely, the Pixy Cam's higher accuracy 83.3% and stable performance confirm the effectiveness of its integrated image processing. The visual confirmation from PixyMon, showing simultaneous bounding boxes for Red, Green, and Blue, is crucial: it proves the Pixy Cam's ability to provide spatial intelligence (Position and Size data) and multi-object tracking, capabilities entirely absent from the Color Sensor's output. This means the Pixy Cam is indispensable for complex tasks like robotic sorting or navigation where *where* an object is, is as important as *what* color it is.

Discrepancies Between Expected and Observed Outcomes

While the Pixy Cam performed better overall, one discrepancy was the false positive (misidentifying Green as Red). This observed error, common in image processing, can be attributed to several factors related to the Pixy's color classification system. Since the Pixy stores a color signature as a range of values in a color space (e.g., Hue-Saturation-Value or HSV), the color of a background object or a reflection might have a hue and saturation that falls within the established tolerance range of the taught signature. The phone screen, being reflective, is prone to specular highlights from strong light sources (e.g., room light), which can create a spurious, transient detection block that falls close to the 'Red' signature's tolerance, causing the misidentification. This issue is less about the sensor's physical failure and more about the boundaries set during the initial software training (Color Tolerance).

Sources of Error or Limitations of the Experiment

1. Limited Sample Size: The most significant limitation is the small sample size (n=2 trials per color). The accuracy percentages obtained (66.7% vs. 83.3%) provide a strong directional trend but are not statistically robust.
2. Calibration Dependency: The experiment was highly dependent on the initial calibration. Any minor inaccuracy in setting the software thresholds for the Color Sensor would amplify the "Colour unclear" errors observed.
3. Lack of Stress Testing: The experiment did not quantitatively measure the devices' performance under varying conditions (e.g., changes in ambient light or object distance).

This limits the ability to fully quantify the Pixy Cam's superior robustness, which is one of its primary theoretical advantages.

CONCLUSION

In conclusion, this experiment compared the performance of a conventional color sensor (TCS3200) and the Pixy Cam for color detection in microcontroller-based systems. The findings show that the **Pixy Cam provided more accurate, consistent, and reliable color detection**, particularly in environments with varying lighting conditions. Its ability to supply spatial information such as object position and size significantly enhances its usefulness for advanced applications. In contrast, the TCS3200 performed adequately for simple, close-range color detection but was highly sensitive to calibration and ambient lighting.

The results of the experiment support the hypothesis that a vision-based sensor with onboard image processing would outperform a conventional color sensor in terms of detection reliability and response efficiency. The Pixy Cam's autonomous processing reduced the computational load on the microcontroller while delivering high-level, ready-to-use data.

These findings have broader implications for mechatronic and robotic systems, where accurate object recognition and tracking are essential. Vision-based sensors such as the Pixy Cam are better suited for applications including robotic navigation, automated sorting, and intelligent control systems, while traditional color sensors remain suitable for low-cost, controlled environments.

RECOMMENDATIONS

Several improvements can be implemented to address the limitations identified in this experiment. First, the number of experimental trials should be significantly increased. Conducting a larger sample size, such as at least 50 trials per color, would allow for statistically reliable accuracy calculations and reduce the influence of random variation. This would enable a more robust comparison between the conventional color sensor and the Pixy Cam.

Second, calibration procedures should be refined and standardized. For the color sensor, more precise calibration techniques can be applied by systematically determining threshold values under controlled lighting conditions. Repeating calibration before each testing session would minimize classification errors such as “colour unclear” and improve detection consistency.

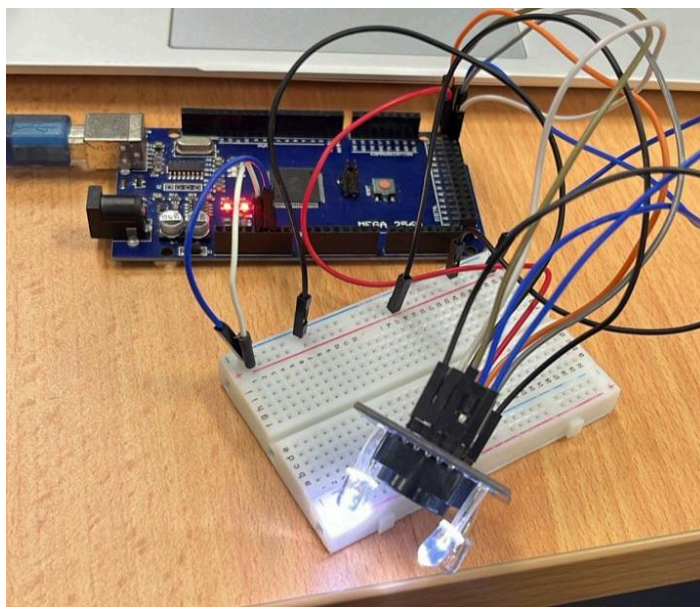
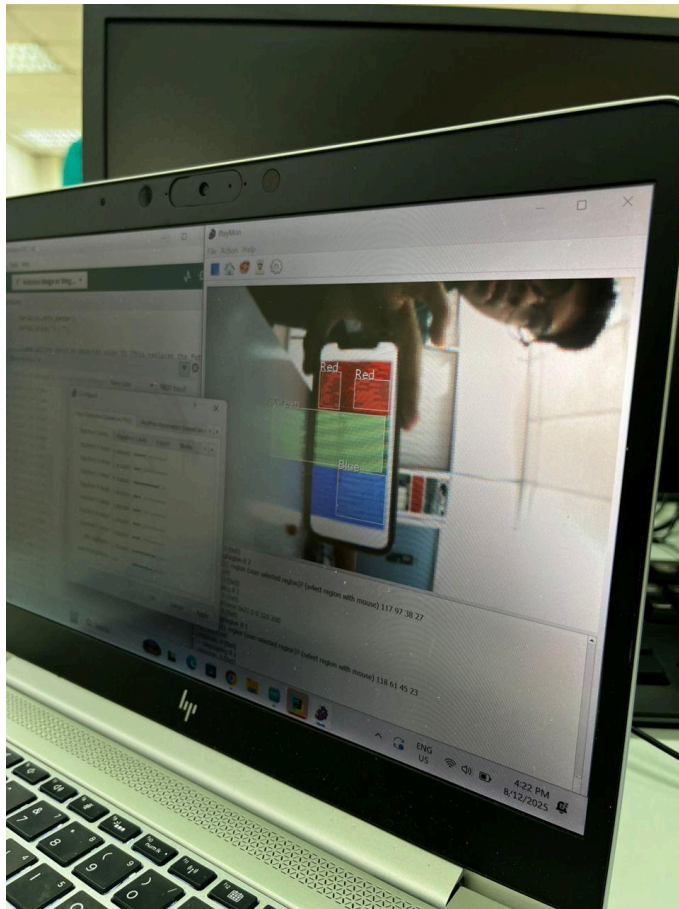
Third, future experiments should incorporate stress testing under varying environmental conditions. Controlled variations in ambient lighting intensity and object distance should be introduced and measured quantitatively. This would allow for a more comprehensive evaluation of each sensor’s robustness and would better demonstrate the Pixy Cam’s ability to maintain accuracy under dynamic conditions.

Finally, implementing automated data logging and real-time performance monitoring would improve experimental reliability. Recording response time, detection accuracy, and misclassification rates programmatically would enhance analysis quality and provide deeper insight into system behavior.

REFERENCES

1.  Pixy2 Camera - Image Recognition for Arduino & Raspberry Pi
2. Arduino. (n.d.). Arduino documentation: Sensor interfacing and serial communication. <https://www.arduino.cc/reference/en/>
3. Charmed Labs. (n.d.). Pixy CMUcam5: A smart vision sensor for color-based object detection. <https://pixycam.com/>
4. Charmed Labs. (n.d.). PixyMon user guide and Pixy CMUcam5 documentation. <https://docs.pixycam.com/>
5. How To Mechatronics. (n.d.). Arduino color sensing tutorial – TCS230/TCS3200 color sensor. <https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial-tcs230-tcs3200-color-sensor/>
6. Mechatronics System Integration. (2024). Week 9: Image and video input interfacing with microcontroller (ver. 2a). MCTA3203 Laboratory Manual.
7. Szeliski, R. (2010). Computer vision: Algorithms and applications. Springer.

APPENDICES



ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahyu Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: *iman*

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature:

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517